Reviewer Pack — Minimal K-theoretic Index and Frege Proof Depth Inequality

Entry 35b64e33f7e5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 04:09:33 UTC

Minimal K-theoretic Index and Frege Proof Depth Inequality

Entry ID: `35b64e33f7e5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 04:09:33 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory (K-theory) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For all CNF formulas φ with n variables, the minimal index of a simple module over the Boolean ring B is at least Ω(n^(2/3)) when the Frege proof depth of φ is at most d.

Rationale (proposer's reasoning):

> The K-theoretic index measures non-triviality in the algebraic sense, which may reflect the complexity of resolving logical statements. A strong correlation between this index and Frege proof depth could provide a novel perspective on the structure of proofs in propositional logic.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fb08358186843cb5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF formula φ with n variables, if the K-theoretic index is ≥ Ω(n^(2/3)) and the Frege proof depth is ≤ d.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Algebraic K-theory' AND 'Frege proof complexity' AND 'index inequality' - 'K-theoretic index' AND 'Boolean satisfiability' AND 'proof depth' - 'minimal index' INALPHA 'simple module' AND 'CNF formulas' AND 'Frege proof depth'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for i in range(n) for j in range(i+1, n)):
                clauses.append(clause)
        return clauses

    def frege_proof_depth(cnf):
        # Simplified DPLL solver to estimate proof depth
        stack = []
        for clause in cnf:
            if all(x not in stack and -x not in stack for x in clause):
                stack.extend(clause)
            else:
                continue  # This is a simplified heuristic
        return len(stack)

    def k_theoretic_index(cnf):
        n = len(cnf[0])
        B = [[0] * n for _ in range(n)]
        for clause in cnf:
            for x in clause:
                if x > 0:
                    B[x-1][x-1] += 1
                else:
                    B[-x-1][-x-1] += 1
        det = determinant(B)
        return abs(det) ** (1/n)

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
```

```
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * determinant(submatrix)
        return det

    def is_valid_cnf(cnf, n):
        if not all(len(clause) > 0 for clause in cnf):
            return False
        if any(abs(x) > n for x in [x for clause in cnf for x in clause]):
            return False
        return True

    results = []
    for n in range(5, 41):
        for _ in range(30):  # Ensure at least 30 instances per seed
            cnf = generate_cnf(n)
            if not is_valid_cnf(cnf, n):
                continue
            depth = frege_proof_depth(cnf)
            index = k_theoretic_index(cnf)
            results.append((n, depth, index))

    metric_value = sum(index for _, _, index in results) / len(results)
    conjecture_holds = all(depth <= 10 and index >= n**(2/3) for _, depth, index in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "K-theoretic Index",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "n_max": max(n for n, _, _ in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **result}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21449
2	propose	ollama_remote	glm4:latest	0	0	13074
3	preregistration	ollama_remote	glm4:latest	0	0	9063
4	novelty	ollama_remote	glm4:latest	0	0	8611
5	novelty	ollama_remote	glm4:latest	0	0	8752
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39517
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9855
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15322
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11041
10	judge	ollama_remote	glm4:latest	0	0	12143

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 148827 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/35b64e33f7e5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/35b64e33f7e5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/35b64e33f7e5.tar.gz` (if generated)